Complementary Reinforcement Learning

Dilxat Muhtar, Jiashun Liu, Wei Gao, Weixun Wang, Shaopan Xiong, Ju Huang et al. | alibaba-inc

https://arxiv.org/abs/2603.17621

Research Questions

- How can policy actors and experience extractors co-evolve together during RL training to maintain alignment as capabilities improve?
- What training framework enables efficient asynchronous coordination between actor training and experience distillation at scale?
- How does co-evolutionary experience management compare to static experience approaches in multi-task RL settings?

Core Idea

Existing RL approaches for LLM agents suffer from static experience management - the experience distilled from historical trajectories becomes stale as the actor improves, creating misalignment. The key insight is to co-evolve both the policy actor and experience extractor: the actor optimizes on outcome rewards while the experience extractor optimizes based on whether its distilled experiences actually help the actor succeed. This creates a mutually reinforcing loop where better policies generate higher-quality trajectories that refine the experience bank, which in turn provides more effective guidance for policy optimization.

Key Formula

Split GRPO Objective

$$J_{GRPO}^{split}(\theta) = E\left[\frac{1}{2}\sum_{c\in\{m,\varnothing\}}^{\square}\frac{1}{K_c}\sum_{k=1}^{K_c} L_{clip}(\rho_c, \hat{A}_c)\right]$$

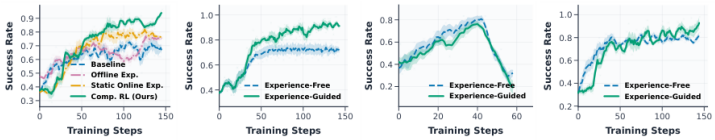
Novel modification to GRPO that computes advantages \hat{A}_c within each subgroup c (experience-guided m vs experience-free ∅) using subgroup-specific means and standard deviations. This prevents bias from differing reward scales between conditions and enables balanced gradient contributions from both subgroups.

CISPO Experience Extractor Objective

$$J_{CISPO}(\phi) = E\left[\frac{\sum_{i=1}^{O}\sum_{t=1}^{|m_i|} sg\left([\rho_{i,t}]_{1-\epsilon_{low}^{IS}}^{1+\epsilon_{high}^{IS}}\right)\hat{A}_i \log \pi_\phi(m_{i,t}|g_i, \tau_i, m_{i,<t})}{\sum_{i=1}^{B}|m_i|}\right]$$

Optimizes experience extractor π_φ using clipped importance sampling ratios ρ_{i,t} and batch-level advantages \hat{A}_i = r(m_i) - \bar{r} where r(m_i) ∈ {-1,+1} indicates whether experience m_i helped actor succeed. The clipping mechanism prevents abrupt distribution shifts during co-evolution.

Key Figure



(a) Exp. Comparison (b) w/o Group Split (c) Cross-group Adv. (d) Subgroup Adv.

Figure 3: **(a)** Co-evolving the actor and experience extractor consistently outperforms static alternatives. **(b–d)** Ablation study on advantage estimation designs for the actor. The Exp. denotes experience.

2 Methodology

2.1 Problem Formulation

We consider an LLM-based actor π_θ operating in an interactive environment $\mathcal{E}$, formalized as a Markov Decision Process (MDP) $\langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R} \rangle$ (Silver & Veness, 2010), where $\mathcal{S}$, $\mathcal{A}$ are the state and action spaces, $\mathcal{T} : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$ is the transition function, and $\mathcal{R} : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the reward function. At the beginning of each episode, the agent receives a task goal g. At each timestep t, it receives an observation $s_t \in \mathcal{S}$, produces an internal reasoning trace by reflecting on the current observation and interaction history, and then decides an action $a_t \sim \pi_\theta(\cdot \mid s_{\leq t}, g)$ (Yao et al., 2022). The environment then transitions to the next state s_{t+1}. An episode terminates upon task completion or upon reaching $T_{\max}$ steps, yielding a outcome reward $R \in \{0, 1\}$. The objective is to maximize the expected success rate across diverse tasks and environments:

$$\mathcal{J}(\theta) = \mathbb{E}_{\mathcal{E}, g, \tau \sim \pi_\theta} \left[R(\tau) \right], \tag{1}$$

where $\tau = (s_0, a_0, s_1, a_1, \ldots, s_T)$ denotes the full interaction trajectory.

The formulation above treats each trajectory τ in isolation, optimizing π_θ solely from binary outcome rewards, leaving the rich behavioral information embedded in each trajectory unexploited. A natural path toward greater learning efficiency is to distill structured experience m from past trajectories, store it in an experience bank $\mathcal{M}$, and retrieve relevant entries to guide π_θ in subsequent episodes (Silver & Sutton; Ouyang et al., 2025; Zhang et al., 2026a; Zhai et al., 2025). This augments the original objective (Equation 1) to:

$$\mathcal{J}(\theta) = \mathbb{E}_{\mathcal{E}, g, m \sim \mathcal{M}, \tau \sim \pi_\theta(\cdot \mid g, m)} \left[R(\tau) \right]. \tag{2}$$

2.2 From Static to Co-Evolutionary Experience

Having formalized the learning-from-experience framework, we now turn to answering a practical question: how should the experience bank $\mathcal{M}$ be constructed and maintained to maximally benefit actor learning? We analyze three design choices through a pilot study on the MiniHack Room (Samvelyan et al., 2021) [1] : **(1) Baseline**: learning without experience; **(2) Offline Exp.**: $\mathcal{M}$ is pre-constructed from prior collected trajectories using an external extractor (Zhai et al., 2025) and remains static during RL training; **(3) Static Online Exp.**: $\mathcal{M}$ is dynamically maintained by a fixed experience extractor π_ϕ during actor learning. Figure 3a shows that while offline experience provides an initial performance boost, its benefit decays progressively over the course of training. Similarly, static online experience yields only marginal gains over the baseline, suggesting that simply collecting online experience without co-evolving the extractor is insufficient. We attribute this to a **distributional misalignment**: a static $\mathcal{M}$ cannot track the evolving state-action distribution of π_θ, causing the guidance to become stale and counterproductive. This insight motivates us to the *co-evolutionary* paradigm where π_ϕ and π_θ are jointly optimized. In this framework, improved policies generate higher-quality trajectories that refine $\mathcal{M}$, thereby providing more effective guidance for subsequent policy optimization. We formalize this mutually reinforcing mechanism as `Complementary RL`.

2.3 Complementary Reinforcement Learning

Algorithm Design for Experience Extractor In `Complementary RL`, the experience bank $\mathcal{M}$ is maintained by an experience extractor π_ϕ, which is jointly optimized with the actor π_θ. At the end of each episode, the extractor distills an experience entry $m \sim \pi_\phi(\cdot \mid g, \tau)$ conditioned on the task goal g and the full interaction trace τ. We track how m influences subsequent actor behavior by assigning a binary

[1] `Room-Ultimate-5x5-v0`: `minihack-room`

3

Figure 3: Demonstrates that co-evolving actor and experience extractor consistently outperforms static alternatives, and shows the critical importance of split advantage estimation for stable training.

Limitations

- Requires separate parameter sets for actor and experience extractor, increasing computational overhead
- Performance depends on quality of experience distillation, which may be unreliable for weak base models or complex tasks

- Asynchronous training framework adds implementation complexity compared to standard RL approaches
- Limited evaluation on tasks beyond the four environments tested (MiniHack, WebShop, ALFWorld, SWE-Bench)